

MatML — Complete Professional Engineering Guide

Material Property Prediction using Machine Learning

End-to-End Implementation Manual

Accent Color: Emerald Green (#10b981)

Design Rationale: Emerald green is the deliberate choice for MatML. It references lab equipment LEDs, circuit boards, and oscilloscope displays — the visual language of scientific instrumentation. Against a near-black background, it creates maximum contrast without harshness, signaling precision and technical authority. It avoids the cliché blue of "generic tech" and the red of "danger", landing exactly at the intersection of scientific and modern.

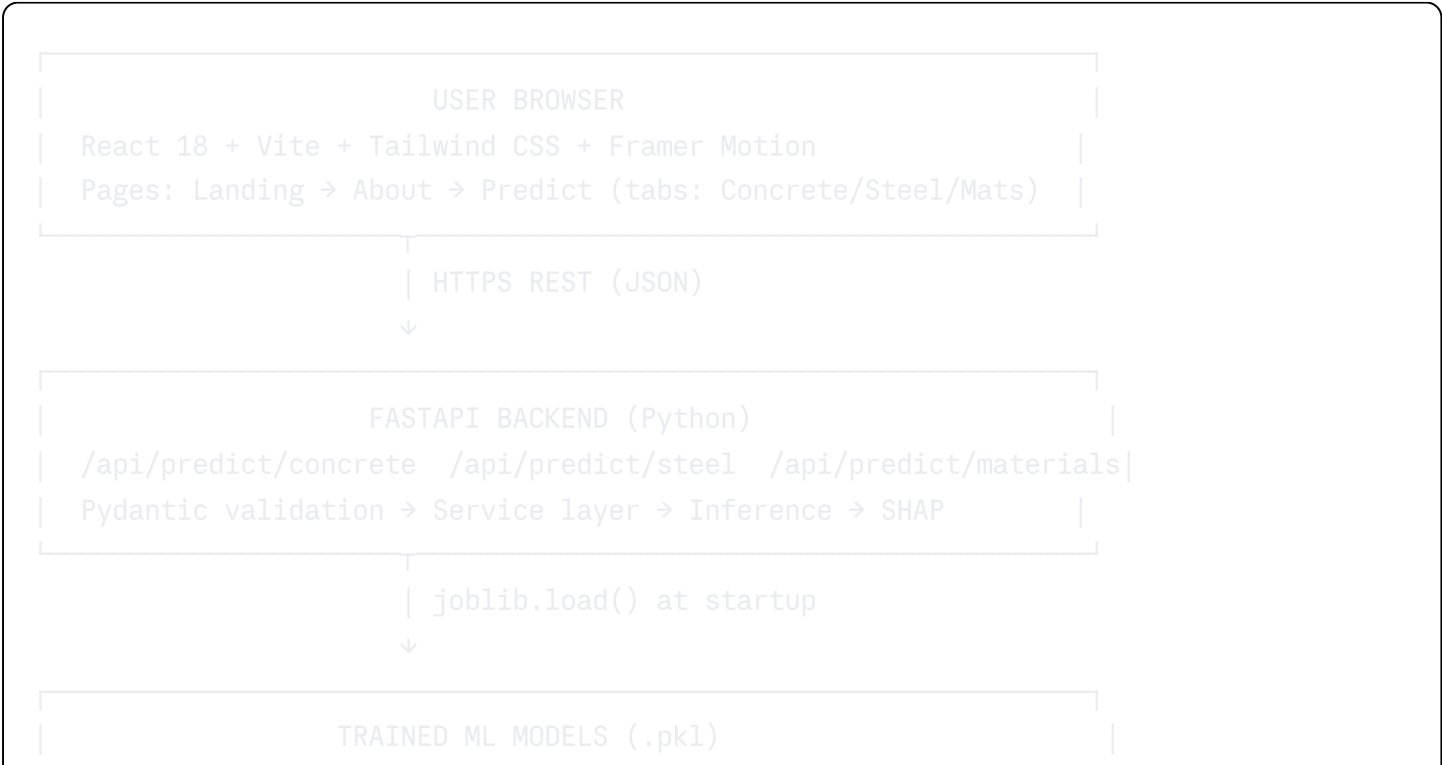
1. OVERVIEW

Project Purpose

MatML is a full-stack machine learning web application that predicts mechanical properties of engineering materials — concrete, steel alloys, and general material alloys — from their composition or mechanical descriptors, without requiring physical laboratory testing. It serves engineering students, materials researchers, and industry professionals who need rapid, explainable property estimates during the design phase.

System Architecture

High-Level Architecture



```
| concrete_best_model.pkl  concrete_scaler.pkl  |
| steel_best_model.pkl    steel_scaler.pkl    |
| materials_reg_model.pkl  materials_cls_model.pkl |
| materials_scaler.pkl    |
```

Detailed Data Flow

```
User fills form (e.g., Cement=540, Water=162, Age=28...)
  → InputField components with ghost text + tooltips
  → Zustand store updates formData
  → User clicks "Predict"
  → usePrediction hook calls predictions.js → predictConcrete()
  → fetch POST /api/predict/concrete with JSON body
  → FastAPI validates with ConcreteInput Pydantic schema
  → concrete_service.py calls inference.py
  → Loads concrete_best_model + scaler from app.state.models
  → scaler.transform(input) → model.predict() → SHAP values
  → Returns ConcretePrediction JSON
  → React PredictionResult renders glass result cards
  → ShapChart renders horizontal SHAP bar chart
```

Tech Stack Justification

Layer	Choice	Reason
Frontend framework	React 18 + Vite	Fast HMR, massive ecosystem, component reuse
Styling	Tailwind CSS	Utility-first — perfect for consistent design tokens
Animation	Framer Motion	Production-grade, declarative, React-native
State	Zustand	1/10th the boilerplate of Redux, sufficient for this app
Backend	FastAPI	Native Python, async, auto OpenAPI docs, Pydantic integration
ML	scikit-learn + XGBoost	Industry standard, proven on tabular data
Explainability	SHAP	Gold standard for tree model explainability
Model serialization	joblib	Faster than pickle for numpy-heavy objects
Training env	Anaconda	Isolated Python env, Jupyter support

Layer	Choice	Reason
Deployment	Render.com	Free tier, Docker support, simple env var management

Why NOT Node.js backend: ML models are Python objects. A Node.js intermediary would add latency, complexity, and a translation layer with no benefit. FastAPI talks directly to Python models — zero overhead.

2. PROJECT STRUCTURE

Full Monorepo Layout

matml/	← Monorepo root
├─ CLAUDE.md	← Antigravity master context
├─ .claude/	← Antigravity agent definitions
├─ SKILLS.md	
├─ agents/	
├─ frontend-agent.md	
├─ backend-agent.md	
├─ ml-pipeline-agent.md	
├─ frontend/	← React application
├─ public/	
├─ favicon.svg	← Custom molecular/hex icon
├─ src/	
├─ components/	
├─ ui/	← Reusable primitives
├─ GlassCard.jsx	← Core glass container
├─ GlassNavbar.jsx	← Fixed floating navbar
├─ InputField.jsx	← Input + ghost text + tooltip
├─ TooltipIcon.jsx	← ⓘ hover tooltip
├─ ResultCard.jsx	← Metric display card
├─ ModelBadge.jsx	← "RandomForest" pill badge
├─ TabSwitcher.jsx	← Concrete/Steel/Materials tabs
├─ LoadingSpinner.jsx	← Emerald pulse spinner
├─ sections/	← Landing page sections
├─ HeroSection.jsx	← Hero + gradient orbs + CTA
├─ FeaturesSection.jsx	← 3 feature glass cards
├─ StatsSection.jsx	← "3 Materials 6 Models" row
├─ CTASection.jsx	← Bottom redirect banner
├─ predict/	← Prediction page components
├─ ConcreteForm.jsx	← 8 input fields
├─ SteelForm.jsx	← 13 element input fields

- └─ MaterialsForm.jsx ← 4 mechanical property fields
 - └─ PredictionResult.jsx ← Result glass cards grid
 - └─ ShapChart.jsx ← Recharts SHAP bar chart
 - └─ ModelComparisonTable.jsx ← Model metrics table
 - └─ pages/
 - └─ LandingPage.jsx ← Full scroll landing
 - └─ AboutPage.jsx ← Project + model info
 - └─ PredictPage.jsx ← Tabbed prediction interface
 - └─ api/
 - └─ predictions.js ← ALL fetch calls – single source
 - └─ hooks/
 - └─ usePrediction.js ← Prediction async logic
 - └─ store/
 - └─ predictionStore.js ← Zustand global state
 - └─ utils/
 - └─ validators.js ← Form validation rules
 - └─ styles/
 - └─ globals.css ← Font imports + base resets
 - └─ App.jsx ← Router + layout
 - └─ main.jsx ← React entry point
 - └─ index.html ← Google Fonts links here
 - └─ vite.config.js
 - └─ tailwind.config.js
 - └─ .env ← VITE_API_URL=http://localhost:8000
 - └─ package.json
- └─ backend/ ← FastAPI application
 - └─ app/
 - └─ __init__.py
 - └─ main.py ← App factory, CORS, lifespan, routers
 - └─ config.py ← Pydantic Settings from .env
 - └─ routers/
 - └─ __init__.py
 - └─ concrete.py ← POST /api/predict/concrete
 - └─ steel.py ← POST /api/predict/steel
 - └─ materials.py ← POST /api/predict/materials
 - └─ schemas/ ← Pydantic input/output models
 - └─ __init__.py
 - └─ concrete.py ← ConcreteInput, ConcretePrediction
 - └─ steel.py ← SteelInput, SteelPrediction
 - └─ materials.py ← MaterialsInput, MaterialsPrediction
 - └─ services/ ← Business logic layer
 - └─ __init__.py
 - └─ concrete_service.py
 - └─ steel_service.py
 - └─ materials_service.py
 - └─ ml/ ← ML model management

```

├── ├── ├── ├── __init__.py
├── ├── ├── ├── loader.py           ← Load all .pkl at startup
├── ├── ├── ├── inference.py       ← Predict + SHAP computation
├── ├── ├── ├── utils/
├── ├── ├── ├── ├── logger.py      ← Structured logging setup
├── ├── ├── ├── trained_models/   ← .pkl files (committed after training)
├── ├── ├── ├── ├── concrete_best_model.pkl
├── ├── ├── ├── ├── concrete_scaler.pkl
├── ├── ├── ├── ├── steel_best_model.pkl
├── ├── ├── ├── ├── steel_scaler.pkl
├── ├── ├── ├── ├── materials_reg_model.pkl
├── ├── ├── ├── ├── materials_cls_model.pkl
├── ├── ├── ├── ├── materials_scaler.pkl
├── ├── ├── ├── requirements.txt
├── ├── ├── ├── .env               ← FRONTEND_URL, LOG_LEVEL
├── ├── ├── ├── Dockerfile
├── ├──
├── ├── training/                 ← Standalone training scripts
├── ├── ├── data/
├── ├── ├── ├── concrete_strength.csv ← Dataset A (~1030 rows)
├── ├── ├── ├── steel_strength.csv   ← Dataset B (thousands of rows)
├── ├── ├── ├── material.csv         ← Dataset C (Materials)
├── ├── ├── ├── notebooks/          ← EDA notebooks
├── ├── ├── ├── ├── 01_concrete_eda.ipynb
├── ├── ├── ├── ├── 02_steel_eda.ipynb
├── ├── ├── ├── ├── 03_materials_eda.ipynb
├── ├── ├── ├── train_concrete.py    ← Trains + saves concrete models
├── ├── ├── ├── train_steel.py       ← Trains + saves steel models
├── ├── ├── ├── train_materials.py   ← Trains + saves materials models
├── ├── ├── ├── requirements_training.txt
├── ├──
├── ├── docker-compose.yml          ← Runs frontend + backend together
├── ├── .env.example                ← Template for env vars
├── ├── README.md

```

Naming Convention Table

Layer	Convention	Example
React components	PascalCase	<code>GlassCard</code> , <code>ConcreteForm</code>
React hooks	camelCase <code>use</code> prefix	<code>usePrediction</code>
API functions	camelCase	<code>predictConcrete()</code>
FastAPI routers	snake_case	<code>concrete.py</code>

Layer	Convention	Example
Pydantic models	PascalCase	ConcreteInput, SteelPrediction
API routes	kebab-case	/api/predict/concrete
Python variables	snake_case	yield_strength, best_model
.pkl files	{dataset}_{type}.pkl	concrete_best_model.pkl
Tailwind classes	utility-only	bg-white/5 backdrop-blur-xl

3. MODEL TRAINING (VERY DETAILED)

Dataset Overview

Dataset A — Concrete Compressive Strength

- **Source:** UCI Machine Learning Repository
- **File:** concrete_strength.csv
- **Rows:** ~1,030 | **Columns:** 9 (all numeric)
- **Inputs:** Cement, Blast Furnace Slag, Fly Ash, Water, Superplasticizer, Coarse Aggregate, Fine Aggregate, Age (day)
- **Target:** Concrete compressive strength(MPa, megapascals)
- **Task:** Single-output regression
- **Quality:** Clean — zero missing values

Dataset B — Steel Alloy Composition

- **File:** steel_strength.csv
- **Rows:** Thousands | **Columns:** ~17
- **Inputs:** c, mn, si, cr, ni, mo, v, n, nb, co, w, al, ti (elemental weight percentages)
- **Drop:** formula (text column — no ML signal)
- **Targets:** yield strength, tensile strength, elongation
- **Task:** Multi-output regression (3 targets simultaneously)

Dataset C — Materials Mechanical Properties

- **File:** material.csv
- **Inputs:** E (Young's Modulus), G (Shear Modulus), mu (Poisson's Ratio), Ro (Density)
- **Drop:** Material (text)

- **Regression Targets:** $\overline{Su}$ (Ultimate Tensile Strength), $\overline{Sy}$ (Yield Strength)
 - **Classification Target:** $\overline{Use}$ (TRUE/FALSE $\rightarrow$ 1/0) — multiple material types
 - **Task:** Multi-output regression + binary classification
-

3.1 Data Handling

Loading and Initial Inspection

```
python

import pandas as pd

df = pd.read_csv('data/concrete_strength.csv')

# Always run these 4 checks immediately after loading
print("Shape:", df.shape)                # (rows, columns)
print("Columns:", df.columns.tolist())    # verify exact column names
print("dtypes:\n", df.dtypes)            # check for 'object' types
print("Missing:\n", df.isna().sum())      # count nulls per column
print("Stats:\n", df.describe())          # range, mean, std per column
```

Cleaning Strategies Per Dataset

Concrete: No cleaning needed. Fully numeric, no nulls. Skip to split.

Steel:

```
python

# Step 1: Drop text columns
df = df.drop(columns=['formula'])

# Step 2: Check for remaining object columns
obj_cols = df.select_dtypes(include='object').columns.tolist()
df = df.drop(columns=obj_cols)

# Step 3: Handle missing values (competition data has very few)
df = df.fillna(df.mean(numeric_only=True))
# OR: df = df.dropna() - if missing < 1% of rows
```

Materials:

```
python
```

```
# Step 1: Drop text column
df = df.drop(columns=['Material'])

# Step 2: Encode boolean target
df['Use'] = df['Use'].map({'TRUE': 1, 'FALSE': 0, True: 1, False: 0}).fillna(0).astype(int)

# Step 3: Fill remaining missing values
df = df.fillna(df.mean(numeric_only=True))
```

Handling Missing Values — Decision Framework

Missing %	Strategy
0%	Nothing needed
< 5%	<code>fillna(column_mean)</code>
5-20%	<code>fillna(median)</code> or row drop if pattern is random
> 20%	Consider dropping column; never impute blindly

3.2 Feature Engineering

Concrete — Feature Insights

- **Age** is the most powerful feature (nonlinear effect — strength grows fast early, flattens)
- **Water/Cement ratio** is a classic materials science metric — consider creating it
- **Superplasticizer** interacts with Water — reducing water need
- Optional engineered feature: `wc_ratio = Water / Cement`

Steel — Feature Selection

- All 13 elemental composition features are domain-relevant — keep all
- `formula` column is a string identifier — drop, carries no numeric signal
- No dimensionality reduction needed — 13 features is manageable
- Carbon (`C`) and Chromium (`Cr`) expected to dominate feature importance

Materials — Low Dimensionality

- Only 4 input features (E, G, mu, Ro) — no reduction needed
- These are fundamental mechanical property descriptors — all retained

3.3 Model Selection

Why These Models

LinearRegression (Baseline — all modules)

- Runs in milliseconds, fails loudly on nonlinear data
- Required academically: proves RF/XGB improvement is earned, not random
- If LR $R^2=0.62$ and RF $R^2=0.91 \rightarrow$ nonlinearity confirmed, complex model justified

RandomForestRegressor / RandomForestClassifier (Main Workhorse)

- Handles nonlinear feature interactions automatically
- Example: effect of Water depends on Cement amount — RF captures this
- Built-in feature importance: reveals which ingredients dominate
- Robust to outliers and skewed distributions
- Works for both regression AND classification with zero code changes
- Consistent, reliable — no hyperparameter sensitivity

XGBoostRegressor / MultiOutputRegressor(XGB) (Performance Upgrade)

- Gradient-boosted trees — consistently outperform RF on larger tabular datasets
- L1/L2 regularization prevents overfitting on noisy real-world data
- Faster inference than RF for large models
- Used for Steel (large dataset, high-dimensional) and Materials regression

MultiOutputRegressor (Wrapper — not a model)

- scikit-learn wrapper that runs any single-output model once per target column
- Enables simultaneous prediction of 3 steel properties from one `.predict()` call
- Wraps RF or XGB — compare both and pick the winner

Model Allocation

Module	Baseline	Main	Upgrade	Task
Concrete	LinearRegression	RandomForest	XGBoost	Single regression
Steel	LR (per target)	MultiOutput(RF)	MultiOutput(XGB)	Multi regression

Module	Baseline	Main	Upgrade	Task
Materials reg	LR (per target)	MultiOutput(RF)	—	Multi regression
Materials cls	—	RandomForestClassifier	—	Binary classification

3.4 Training Pipeline

Universal 10-Step Pipeline (applied to all 3 modules)

```

Step 1: Load CSV with pd.read_csv()
Step 2: Inspect – shape, dtypes, missing, describe()
Step 3: Clean – drop text/ID cols, handle missing values
Step 4: Split X (features) and y (targets)
Step 5: Train/test split – 80/20, random_state=42
Step 6: Scale features – StandardScaler fit on TRAIN only
Step 7: Train baseline – LinearRegression
Step 8: Train main model – RandomForest / MultiOutputRegressor
Step 9: Evaluate – MAE, R², compare all models, print table
Step 10: Save winner + scaler as .pkl files

```

Evaluation Metrics

Regression metrics:

- **MAE (Mean Absolute Error):** Average prediction error in MPa — interpretable, same unit as target
- **RMSE (Root Mean Squared Error):** Penalizes large errors more — useful for safety-critical predictions
- **R² (Coefficient of Determination):** Proportion of variance explained. R²=1.0 is perfect; R²=0 means model predicts the mean; negative means model is worse than the mean
- **Target R² thresholds:** >0.85 = good; >0.90 = excellent; >0.95 = exceptional

Classification metrics:

- **Accuracy:** Proportion correct — sufficient when classes are balanced
- **F1-Score:** Harmonic mean of precision and recall — better when classes imbalanced
- **Confusion Matrix:** Shows exact true/false positive/negative counts

Cross-Validation

```
python

from sklearn.model_selection import cross_val_score

# 5-fold CV on training set
cv_scores = cross_val_score(model, X_train_sc, y_train, cv=5, scoring='r2')
print(f"CV R²: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")
# Std > 0.05 suggests overfitting or unstable model
```

3.5 Hyperparameter Tuning

RandomForest Key Parameters

```
python

RandomForestRegressor(
    n_estimators=200,          # More trees = more stable; diminishing returns after 200
    max_depth=None,           # Let trees grow fully – RF's built-in bagging prevents overfitting
    min_samples_split=2,      # Increase to 5-10 if overfitting
    min_samples_leaf=1,       # Increase to 2-4 if overfitting
    max_features='sqrt',      # Default for regression – good starting point
    random_state=42,
    n_jobs=-1                 # Use all CPU cores
)
```

XGBoost Key Parameters

```
python

XGBRegressor(
    n_estimators=200,          # Number of boosting rounds
    max_depth=5,              # Deeper = more complex; 4-6 is typical
    learning_rate=0.05,       # Lower = slower but more robust; use more estimators
    subsample=0.8,            # Row sampling per tree – prevents overfitting
    colsample_bytree=0.8,     # Feature sampling per tree
    gamma=0,                  # Min loss reduction to split; increase to regularize
    reg_alpha=0,              # L1 regularization
    reg_lambda=1,             # L2 regularization
    random_state=42,
    verbosity=0
)
```

Quick Tuning (if time allows — use GridSearch on key params)

```
python

from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [4, 6, None],
}

grid = GridSearchCV(RandomForestRegressor(random_state=42), param_grid, cv=3, scoring='r
grid.fit(X_train_sc, y_train)
print("Best params:", grid.best_params_)
print("Best R²:", grid.best_score_)
```

3.6 Model Export

Saving Strategy (always save both model AND scaler)

```
python

import joblib
import os

OUTPUT_DIR = '../backend/trained_models'
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Save winner
joblib.dump(best_model, f'{OUTPUT_DIR}/concrete_best_model.pkl')
joblib.dump(scaler, f'{OUTPUT_DIR}/concrete_scaler.pkl')

# Verify by loading back
loaded = joblib.load(f'{OUTPUT_DIR}/concrete_best_model.pkl')
test_pred = loaded.predict(X_test_sc[:1])
print(f"Verification prediction: {test_pred}")
```

Why joblib over pickle

- joblib uses memory-mapped numpy arrays — 10-100x faster for large arrays
- RandomForest models contain many numpy arrays — critical advantage
- `joblib.dump(model, file, compress=3)` adds compression if file size is large

3.7 Expected Training Outputs

After running all three training scripts, `backend/trained_models/` should contain:

```
concrete_best_model.pkl    (~5-15 MB for RF with 200 trees)
concrete_scaler.pkl        (<1 MB)
steel_best_model.pkl       (~20-50 MB for MultiOutput XGB)
steel_scaler.pkl           (<1 MB)
materials_reg_model.pkl    (~5-10 MB)
materials_cls_model.pkl    (~5-10 MB)
materials_scaler.pkl       (<1 MB)
```

4. BACKEND DEVELOPMENT

Framework: FastAPI

FastAPI is the correct choice because: it's pure Python (same language as ML stack), provides automatic OpenAPI documentation at `/docs`, uses async by default (non-blocking I/O), and Pydantic schema validation catches bad input before it reaches the ML model.

Project Setup

```
bash

cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Application Entry Point (`app/main.py`)

```
python
```

```

from contextlib import asynccontextmanager
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from app.routers import concrete, steel, materials
from app.ml.loader import load_all_models
from app.config import settings
from app.utils.logger import setup_logging
import logging

setup_logging()
logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Load models at startup, clean up at shutdown."""
    logger.info("🚀 MatML API starting – loading models...")
    app.state.models = load_all_models()
    loaded = [k for k, v in app.state.models.items() if v is not None]
    logger.info(f"✅ Models loaded: {loaded}")
    yield
    logger.info("🛑 MatML API shutting down")

app = FastAPI(
    title="MatML API",
    description="Material Property Prediction – Concrete, Steel, Materials",
    version="1.0.0",
    lifespan=lifespan
)

# CORS – critical for browser requests from React
app.add_middleware(
    CORSMiddleware,
    allow_origins=[settings.FRONTEND_URL, "http://localhost:5173", "http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["GET", "POST", "OPTIONS"],
    allow_headers=["Content-Type", "Authorization"],
)

# Register routers
app.include_router(concrete.router, prefix="/api/predict", tags=["Concrete"])
app.include_router(steel.router, prefix="/api/predict", tags=["Steel"])
app.include_router(materials.router, prefix="/api/predict", tags=["Materials"])

@app.get("/api/health")
async def health_check(request: Request):

```

```

models = getattr(request.app.state, 'models', {})
loaded = [k for k, v in models.items() if v is not None]
return {"status": "ok", "models_loaded": bool(loaded), "loaded_models": loaded}

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    logger.error(f"Unhandled error: {exc}", exc_info=True)
    return JSONResponse(
        status_code=500,
        content={"status": "error", "message": "Internal server error", "detail": str(exc)}
    )

```

API Design — REST Endpoints

All endpoints follow this pattern:

- Method: `POST` (sending data in body, not query string)
- Content-Type: `application/json`
- Validation: Pydantic schema on input, raises 422 automatically on invalid data
- Response: Structured JSON with `status`, prediction values, `model_used`, `shap_values`

Endpoint Summary

```

POST /api/predict/concrete    ← ConcreteInput → ConcretePrediction
POST /api/predict/steel      ← SteelInput → SteelPrediction
POST /api/predict/materials  ← MaterialsInput → MaterialsPrediction
GET  /api/health             ← health check
GET  /docs                   ← auto-generated Swagger UI (free from FastAPI)

```

Error Handling System

```

python

# In routers — consistent error response format
from fastapi import HTTPException

try:
    result = predict_concrete(model, scaler, input_data.model_dump())
except ValueError as e:
    raise HTTPException(status_code=422, detail=f"Prediction input error: {str(e)}")
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Prediction failed: {str(e)}")

```

Logging System

```
python

# app/utils/logger.py
import logging
import sys

def setup_logging():
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
        handlers=[
            logging.StreamHandler(sys.stdout),
            logging.FileHandler("matml.log")
        ]
    )
```

requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.32.0
pydantic==2.9.0
pydantic-settings==2.6.0
scikit-learn==1.5.2
xgboost==2.1.1
shap==0.46.0
pandas==2.2.3
numpy==1.26.4
joblib==1.4.2
python-multipart==0.0.12
```

5. FRONTEND DEVELOPMENT

Framework and Tooling

- **React 18** with **Vite 5** — fastest dev server, native ES modules
- **Tailwind CSS 3** — utility-first, design tokens in config
- **Framer Motion 11** — production animation library
- **Zustand 4** — minimal global state
- **Recharts 2** — SHAP visualization
- **React Router 6** — client-side routing

Initialization

bash

```
npm create vite@latest frontend -- --template react
cd frontend
npm install tailwindcss postcss autoprefixer framer-motion zustand recharts react-router-dom
npx tailwindcss init -p
```

Design System Implementation

tailwind.config.js

js

```

/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: {
    extend: {
      colors: {
        emerald: {
          DEFAULT: '#10b981',
          hover: '#059669',
          glow: 'rgba(16,185,129,0.15)',
        },
        surface: 'rgba(255,255,255,0.05)',
        'surface-hover': 'rgba(255,255,255,0.08)',
        'border-glass': 'rgba(255,255,255,0.08)',
      },
      fontFamily: {
        display: ['Syne', 'sans-serif'],
        body: ['DM Sans', 'sans-serif'],
        mono: ['JetBrains Mono', 'monospace'],
      },
      backdropBlur: {
        glass: '20px',
      },
      backgroundImage: {
        'gradient-radial': 'radial-gradient(var(--tw-gradient-stops))',
      },
      animation: {
        'float': 'float 6s ease-in-out infinite',
        'glow-pulse': 'glowPulse 3s ease-in-out infinite',
      },
      keyframes: {
        float: {
          '0%, 100%': { transform: 'translateY(0px)' },
          '50%': { transform: 'translateY(-20px)' },
        },
        glowPulse: {
          '0%, 100%': { boxShadow: '0 0 20px rgba(16,185,129,0.2)' },
          '50%': { boxShadow: '0 0 40px rgba(16,185,129,0.4)' },
        },
      },
    },
  },
  plugins: [],
}

```

index.html — Font Imports

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>MatML - Material Property Prediction</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800&family=
</head>
<body class="bg-[#0a0a0a]">
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

Core Components

GlassCard.jsx

jsx

```

import { motion } from 'framer-motion';

export default function GlassCard({ children, className = '', hover = false, animate = true }) {
  const base = `
    bg-white/5 backdrop-blur-xl border border-white/8 rounded-2xl
    ${hover ? 'hover:bg-white/8 hover:border-emerald-DEFAULT/30 transition-all duration-300' : ''}
    ${className}
  `;

  if (!animate) return <div className={base}>{children}</div>;

  return (
    <motion.div
      className={base}
      initial={{ opacity: 0, y: 20 }}
      whileInView={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.5 }}
      viewport={{ once: true }}
    >
      {children}
    </motion.div>
  );
}

```

InputField.jsx

jsx

```

import { useState } from 'react';
import TooltipIcon from './TooltipIcon';

export default function InputField({
  label, name, value, onChange,
  placeholder, tooltip, unit,
  min, max, step = 'any',
  type = 'number', error
}) {
  return (
    <div className="flex flex-col gap-1.5">
      <div className="flex items-center gap-2">
        <label className="text-sm font-medium text-gray-300 font-body">
          {label}
        </label>
        {tooltip && <TooltipIcon text={tooltip} />}
      </div>
      <div className="relative">
        <input
          type={type}
          name={name}
          value={value}
          onChange={onChange}
          placeholder={placeholder}
          min={min} max={max} step={step}
          className={`
            w-full px-4 py-3 rounded-xl font-mono text-sm
            bg-white/5 border backdrop-blur-sm
            text-white placeholder-gray-600
            focus:outline-none focus:ring-2 focus:ring-emerald-DEFAULT/50
            transition-all duration-200
            ${error ? 'border-red-500/50' : 'border-white/10 hover:border-white/20 focus
            ${unit ? 'pr-16' : ''}
          `}
        />
        {unit && (
          <span className="absolute right-3 top-1/2 -translate-y-1/2 text-xs text-gray-5
            {unit}
          </span>
        )}
      </div>
      {error && <p className="text-xs text-red-400">{error}</p>}
    </div>
  );
}

```

jsx

```

import { Link, useLocation } from 'react-router-dom';
import { motion } from 'framer-motion';

export default function GlassNavbar() {
  const { pathname } = useLocation();
  const links = [
    { to: '/', label: 'Home' },
    { to: '/about', label: 'About' },
    { to: '/predict', label: 'Predict' },
  ];

  return (
    <motion.nav
      initial={{ y: -100, opacity: 0 }}
      animate={{ y: 0, opacity: 1 }}
      transition={{ duration: 0.6, ease: 'easeOut' }}
      className="fixed top-4 left-1/2 -translate-x-1/2 z-50 w-[95%] max-w-4xl"
    >
      <div className="bg-white/5 backdrop-blur-xl border border-white/8 rounded-2xl px-6"
        <Link to="/" className="font-display font-bold text-lg text-white flex items-center"
          <span className="text-emerald-DEFAULT">⬅</span>
            MatML
          </Link>
        <div className="flex items-center gap-1">
          {links.map(({ to, label }) => (
            <Link
              key={to}
              to={to}
              className={`px-4 py-2 rounded-xl text-sm font-body transition-all duration
                ${pathname === to
                  ? 'bg-emerald-DEFAULT/20 text-emerald-DEFAULT'
                  : 'text-gray-400 hover:text-white hover:bg-white/5'}
                `}
            >
              {label}
            </Link>
          ))}
        </div>
        <Link
          to="/predict"
          className="px-5 py-2 rounded-xl bg-emerald-DEFAULT text-black text-sm font-medium
            hover:bg-emerald-hover transition-colors duration-200"
        >
          Try Now →
        </Link>
      </div>
    </motion.nav>
  );
}

```

```
    </motion.nav>
  );
}
```

State Management (Zustand)

```
js

// src/store/predictionStore.js
import { create } from 'zustand';

const defaultFormData = {
  concrete: {
    cement: 350, blast_furnace_slag: 0, fly_ash: 0, water: 185,
    superplasticizer: 6, coarse_aggregate: 1000, fine_aggregate: 750, age: 28
  },
  steel: {
    c: 0.1, mn: 0.5, si: 0.3, cr: 0.5, ni: 0.1, mo: 0.2,
    v: 0.01, n: 0.01, nb: 0.01, co: 0.01, w: 0.01, al: 0.01, ti: 0.01
  },
  materials: { E: 200000, G: 77000, mu: 0.3, Ro: 7800 }
};

export const usePredictionStore = create((set) => ({
  activeTab: 'concrete',
  isLoading: false,
  result: null,
  error: null,
  formData: defaultFormData,

  setActiveTab: (tab) => set({ activeTab: tab, result: null, error: null }),
  setLoading: (isLoading) => set({ isLoading }),
  setResult: (result) => set({ result, error: null }),
  setError: (error) => set({ error, result: null }),
  updateFormData: (dataset, field, value) => set((state) => ({
    formData: {
      ...state.formData,
      [dataset]: { ...state.formData[dataset], [field]: value }
    }
  })),
  reset: () => set({ result: null, error: null }),
})));
```

API Integration Layer

```
js

// src/api/predictions.js
const BASE = import.meta.env.VITE_API_URL || 'http://localhost:8000';

async function apiCall(endpoint, body) {
  const res = await fetch(`${BASE}${endpoint}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body),
  });

  const data = await res.json();

  if (!res.ok) {
    throw new Error(data.detail || data.message || `HTTP ${res.status}`);
  }

  return data;
}

export const predictConcrete = (formData) => apiCall('/api/predict/concrete', formData);
export const predictSteel = (formData) => apiCall('/api/predict/steel', formData);
export const predictMaterials = (formData) => apiCall('/api/predict/materials', formData);

export async function checkHealth() {
  const res = await fetch(`${BASE}/api/health`);
  return res.json();
}
```

6. INTEGRATION

Connection Architecture

React (port 5173) —POST JSON—> FastAPI (port 8000)
 <—JSON resp—

VITE_API_URL=http://localhost:8000 (frontend .env)
FRONTEND_URL=http://localhost:5173 (backend .env)

Integration Checklist

Before considering integration complete, verify each item:

```
[ ] Backend: uvicorn running on port 8000, no startup errors in terminal
[ ] Backend: GET /api/health returns {"status": "ok", "models_loaded": true}
[ ] Backend: POST /api/predict/concrete with valid JSON returns prediction
[ ] Backend: POST with invalid data returns 422 with detail message
[ ] Frontend: VITE_API_URL set in frontend/.env
[ ] Frontend: npm run dev shows no console errors on load
[ ] Frontend: Prediction form submits without CORS error
[ ] Frontend: Result appears after successful prediction
[ ] Frontend: Error message shown for invalid input or server error
[ ] Integration: SHAP values appear in ShapChart after prediction
```

Testing the Integration

```
bash
```

```
# Test 1: Health check
```

```
curl http://localhost:8000/api/health
```

```
# Test 2: Concrete prediction
```

```
curl -X POST http://localhost:8000/api/predict/concrete \
  -H "Content-Type: application/json" \
  -d '{"cement": 540, "blast_furnace_slag": 0, "fly_ash": 0, "water": 162,
      "superplasticizer": 2.5, "coarse_aggregate": 1040, "fine_aggregate": 676, "age":
```

```
# Expected: {"status": "success", "predicted_compressive_strength_mpa": ~79.4, ...}
```

```
# Test 3: Steel prediction
```

```
curl -X POST http://localhost:8000/api/predict/steel \
  -H "Content-Type: application/json" \
  -d '{"c": 0.02, "mn": 0.05, "si": 0.05, "cr": 0.01, "ni": 19.7, "mo": 2.95,
      "v": 0.01, "n": 0.0, "nb": 0.01, "co": 15.0, "w": 0.0, "al": 0.15, "ti": 1.55}'
```

```
# Test 4: Materials prediction
```

```
curl -X POST http://localhost:8000/api/predict/materials \
  -H "Content-Type: application/json" \
  -d '{"E": 207000, "G": 79000, "mu": 0.3, "Ro": 7860}'
```

Complete Setup from Scratch

bash

1. Clone / create project root

```
mkdir matml && cd matml
```

2. Initialize frontend

```
npm create vite@latest frontend -- --template react
```

```
cd frontend
```

```
npm install tailwindcss postcss autoprefixer framer-motion zustand recharts react-router-dom
```

```
npx tailwindcss init -p
```

```
echo "VITE_API_URL=http://localhost:8000" > .env
```

```
cd ..
```

3. Initialize backend

```
cd backend
```

```
python -m venv venv
```

```
source venv/bin/activate # Windows: venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
echo "FRONTEND_URL=http://localhost:5173" > .env
```

```
echo "LOG_LEVEL=INFO" >> .env
```

```
cd ..
```

4. Set up training environment (Anaconda)

```
cd training
```

```
conda create -n matml-training python=3.11
```

```
conda activate matml-training
```

```
pip install -r requirements_training.txt
```

```
cd ..
```

Running the System

bash

```
# Terminal 1 – Backend
cd backend
source venv/bin/activate
uvicorn app.main:app --reload --port 8000
# Visit http://localhost:8000/docs for auto API docs
```

```
# Terminal 2 – Frontend
cd frontend
npm run dev
# Visit http://localhost:5173
```

```
# Terminal 3 – Training (Anaconda, run once)
conda activate matml-training
cd training
python train_concrete.py
python train_steel.py
python train_materials.py
# .pkl files appear in backend/trained_models/
# Restart backend to load new models
```

```
# Docker (alternative – runs everything)
docker-compose up --build
```

docker-compose.yml

```
yaml
```

```

version: '3.9'

services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    volumes:
      - ./backend/trained_models:/app/trained_models
    environment:
      - FRONTEND_URL=http://localhost:5173
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000

  frontend:
    build: ./frontend
    ports:
      - "5173:80"
    environment:
      - VITE_API_URL=http://localhost:8000
    depends_on:
      - backend

```

8. DEBUGGING & TESTING

Common Issues and Fixes

Error	Root Cause	Fix
<code>CORS policy blocked request</code>	CORS not configured or wrong origin	Add <code>localhost:5173</code> to <code>allow_origins</code> in FastAPI
<code>ValueError: could not convert string to float</code>	Text column left in X	<code>df.select_dtypes(include='object')</code> and drop all
<code>KeyError: 'Concrete compressive strength...'</code>	Wrong column name (case/space)	<code>print(df.columns.tolist())</code> — copy exact string
<code>FileNotFoundError: concrete_best_model.pkl</code>	Training not run yet	Run <code>python train_concrete.py</code> first
<code>R² is negative</code>	Wrong X/y split — targets included in X	Print <code>X.columns</code> — remove all target columns

Error	Root Cause	Fix
422 Unprocessable Entity	Request body doesn't match Pydantic schema	Check frontend JSON keys match schema field names
Network Error in browser	Backend not running	Start uvicorn, check port 8000
Module 'shap' not found	Not installed	pip install shap
Models loaded: [] on startup	.pkl files missing from trained_models/	Run training scripts
Prediction returns mock values	Model is None (pkl not found)	Verify pkl files exist, restart uvicorn

Debug Workflow (step by step)

1. Check browser console (F12) for JavaScript/network errors

2. Check uvicorn terminal for Python errors

3. Test endpoint directly: curl or http://localhost:8000/docs

4. Verify .pkl files: ls backend/trained_models/

5. Check column names: python -c "import pandas as pd; df=pd.read_csv('data/concrete_strength.csv'); print(df.columns.tolist())"

6. Check Pydantic schema field names match frontend JSON keys

7. Run training script with print statements to identify data issue

Unit Testing (pytest)

python

```
# backend/tests/test_concrete.py
import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health():
    res = client.get("/api/health")
    assert res.status_code == 200
    assert res.json()["status"] == "ok"

def test_concrete_prediction_valid():
    payload = {
        "cement": 540, "blast_furnace_slag": 0, "fly_ash": 0,
        "water": 162, "superplasticizer": 2.5,
        "coarse_aggregate": 1040, "fine_aggregate": 676, "age": 28
    }
    res = client.post("/api/predict/concrete", json=payload)
    assert res.status_code == 200
    data = res.json()
    assert "predicted_compressive_strength_mpa" in data
    assert data["status"] == "success"
    assert 0 < data["predicted_compressive_strength_mpa"] < 200

def test_concrete_invalid_input():
    res = client.post("/api/predict/concrete", json={"cement": -100}) # invalid
    assert res.status_code == 422
```

Run tests:

```
bash
cd backend
pip install pytest httpx
pytest tests/ -v
```

9. DEPLOYMENT

Deployment Architecture

```
GitHub Repository
  ↓
Render.com (auto-deploy on push to main)
```

└─ Web Service: FastAPI backend (matml-api.onrender.com)
└─ Static Site: React frontend (matml.onrender.com)

Backend Deployment on Render

Dockerfile

```
dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Render Web Service Settings

- **Environment:** Docker
- **Root Directory:** backend/
- **Start Command:** uvicorn app.main:app --host 0.0.0.0 --port \$PORT
- **Environment Variables on Render:**

```
FRONTEND_URL = https://matml.onrender.com
LOG_LEVEL = INFO
PYTHON_VERSION = 3.11.0
```

Frontend Deployment on Render

Build for production

```
bash

cd frontend
VITE_API_URL=https://matml-api.onrender.com npm run build
# Output: frontend/dist/
```

Render Static Site Settings

- **Root Directory:** `frontend/`
- **Build Command:** `npm install && npm run build`
- **Publish Directory:** `dist`
- **Environment Variables:**

```
VITE_API_URL = https://matml-api.onrender.com
```

Environment Variables Summary

```
bash

# backend/.env (local)
FRONTEND_URL=http://localhost:5173
LOG_LEVEL=INFO
MODEL_DIR=trained_models

# frontend/.env (local)
VITE_API_URL=http://localhost:8000

# .env.example (commit this – template only, no secrets)
FRONTEND_URL=http://localhost:5173
VITE_API_URL=http://localhost:8000
LOG_LEVEL=INFO
```

Scaling Considerations

- **Model loading:** Models load once at startup into `app.state.models` — no per-request I/O
- **Concurrency:** FastAPI + uvicorn handles concurrent requests; add `--workers 4` for multi-core
- **Memory:** RF models with 200 trees can be 10-50MB each; Render free tier (512MB RAM) is tight — if memory issues arise, reduce `n_estimators=100`
- **Cold start:** Render free tier spins down after 15min — first request may be slow (model loading). Use `/api/health` ping to wake up

10. FINAL NOTES

Best Practices Applied

1. **Separation of concerns:** Training, inference, and serving are completely separate

codebases

2. **Single source of truth:** All API calls through `src/api/predictions.js` only
3. **Design tokens:** All colors/fonts in `tailwind.config.js` — change once, updates everywhere
4. **Scaler discipline:** `fit_transform` on train only — enforced in training scripts
5. **Mock-first development:** Backend returns mock responses before .pkl files exist — frontend can develop independently
6. **Fail loud:** Missing .pkl files log warnings but don't crash the server — `/api/health` reports loading status clearly

Optimization Tips

- **SHAP computation is slow for large inputs:** Cache SHAP explainer objects, reuse per model — don't recreate on each request
- **Feature importance caching:** Store R^2 , MAE at training time in a `model_metadata.json` file — load alongside .pkl for the About page
- **React performance:** Use `React.memo` on form components — they shouldn't re-render when result state changes
- **Framer Motion:** Use `viewport={{ once: true }}` on scroll animations — elements don't re-animate on scroll back

Future Improvements (V2 Roadmap)

1. **Target Performance Toggle:** User inputs "I need $S_u \geq 400$ MPa" → optimizer finds minimum-cost composition (`scipy.optimize`)
2. **What-if Sliders:** Real-time prediction on slider move (debounced, 300ms)
3. **Prediction History:** LocalStorage or backend DB stores last 10 predictions per session
4. **Model Comparison Page:** Interactive chart comparing LR vs RF vs XGB across all datasets
5. **Export Results:** Download prediction + SHAP chart as PDF report
6. **Material Recommendation:** Input target properties → classify + recommend material type

End of MatML Engineering Guide — Version 1.0

Built with React + FastAPI + scikit-learn + XGBoost + SHAP

Designed for: College submission | Portfolio | Real-world deployment